AEGIS OS

Agents Component

Component Interface Catalog (CIC)

Contracts, Requirements & Downstream Data Agreements

Document ID	DOC-AEGIS-AGENTS-CIC-001
Version	1.5
Status	Draft
Date	March 2026
Component	Agents Component
Parent System	Aegis OS
Classification	Internal / Design

CONFIDENTIAL — Internal Design Document

Interface Overview

This section defines the interface contracts and downstream requirements for all five Aegis OS components as they relate to the Agents Component. The architecture is hub-and-spoke: the Orchestrator is the sole external counterpart of the Agents Component. Credential Vault, Memory/Storage, and User I/O are reached exclusively through the Orchestrator. The Communications Component provides the physical transport layer.

Architectural principle: the Agents Component knows nothing about Credential Vault, Memory/Storage, or User I/O. It publishes requests to the Orchestrator and receives results back. How the Orchestrator fulfils those requests is entirely the Orchestrator team's responsibility. This section documents what we require the Orchestrator to fulfil on our behalf.

v1.5 change (ADR-004): The Credential Vault now executes credentialed operations on behalf of agents and returns operation results. Raw credential values never leave the Vault boundary. Three new NATS topics support this model. The 'fetch' credential operation has been removed.

Component	Our Relationship	How We Interact
Orchestrator	DIRECT — sole external peer	All events flow directly to/from Orchestrator via NATS. This is the only component the Agents Component is aware of at the integration boundary.
Communications	INFRASTRUCTURE — transport layer	Provides NATS JetStream. The Agents Component connects to the bus; the bus routes events to/from the Orchestrator. The Agents Component has no business logic relationship with Communications.
Credential Vault	INDIRECT — via Orchestrator ◆ **v1.5: Vault executes operations**	We publish credential.request (authorize/revoke) and vault.execute.request events to the Orchestrator. The Vault executes credentialed operations internally and returns results — never raw credential values. We define what we need so the Orchestrator and Vault teams know what to implement.
Memory/Storage	INDIRECT — via Orchestrator	We publish state.write and state.read.request events to the Orchestrator. The Orchestrator persists and retrieves data via the Memory/Storage Component. We define the data schemas so both teams can align.
User I/O	INDIRECT — via Orchestrator	We publish clarification.request events to the Orchestrator. The Orchestrator routes them to the User I/O Component and returns the user's response. We define the interaction contract so the Orchestrator team knows what to relay.

Interface 1: Orchestrator

The Sole External Integration Point — Direct Bilateral Contract

1.1 Relationship Summary

The Orchestrator is the Agents Component's only external peer. Every event the Agents Component publishes is addressed to the Orchestrator. Every event the Agents Component consumes comes from the Orchestrator. This is a deliberate design choice: the Orchestrator acts as the system coordinator, mediating between the Agents Component and all other Aegis OS services (Vault, Memory, User I/O) on our behalf.

The Orchestrator does not just assign tasks — it also acts as our proxy for credentials, Vault-executed operations, state persistence, and user interactions. The v1.5 change (ADR-004) adds a third coordination role: the Orchestrator routes Vault execute requests and relays results, enabling the Vault to execute credentialed operations internally without ever transmitting raw secrets.

1.2 Complete Bilateral Topic Contract

Inbound: Orchestrator → Agents Component (topics we consume)

Topic	Delivery	Schema Ref	What It Represents
`aegis.agents.task.inbound`	At-least-once	EDD §8.3	New task assignment — the primary driver of all agent activity.
`aegis.agents.capability.query`	At-most-once	EDD §8.4	Query whether we have agents capable of specified skill domains.
`aegis.agents.lifecycle.terminate`	At-least-once	EDD §8.8	Instruction to immediately terminate a specific agent.
`aegis.agents.credential.response`	At-least-once	EDD §8.5	Result of an authorize or revoke credential operation. Orchestrator fetched this from the Vault on our behalf. (fetch operation removed in v1.5)
`aegis.agents.vault.execute.progress` ◆ **NEW v1.5**	At-most-once	EDD §8.5	Progress heartbeat from the Vault during a long-running execute operation. Informational only — losing one does not affect correctness.
`aegis.agents.vault.execute.result` ◆ **NEW v1.5**	At-least-once	EDD §8.5	Final result of a Vault-executed operation. Carries operation output — never a raw credential value.
`aegis.agents.state.write.ack`	At-least-once	EDD §8.6	Confirmation that our state.write was persisted by the Memory/Storage Component via Orchestrator.
`aegis.agents.state.read.response`	At-least-once	EDD §8.6	State records retrieved from Memory/Storage by the Orchestrator in response to our read request.
`aegis.agents.clarification.response`	At-least-once	EDD §8.7	User's response to a clarification question, relayed back by the Orchestrator from User I/O.

Outbound: Agents Component → Orchestrator (topics we publish)

Topic	Delivery	Schema Ref	What It Represents

`aegis.orchestrator.task.accepted`	At-least-once	EDD §8.3	Confirmation of task receipt and agent assignment.
`aegis.orchestrator.task.result`	At-least-once	EDD §8.3	Final task output on completion.
`aegis.orchestrator.task.failed`	At-least-once	EDD §8.3	Task failure with error code, phase (incl. vault_execute), and retryable flag.
`aegis.orchestrator.capability.response`	At-most-once	EDD §8.4	Capability query result: match / partial_match / no_match.
`aegis.orchestrator.agent.status`	At-least-once	EDD §8.8	Agent lifecycle state transition notification.
`aegis.orchestrator.credential.request`	At-least-once	EDD §8.5	Request the Orchestrator to perform a credential lifecycle operation (authorize / revoke) with the Credential Vault. fetch removed — use vault.execute.request instead.
`aegis.orchestrator.vault.execute.request` ◆ **NEW v1.5**	At-least-once	EDD §8.5	Request the Orchestrator to execute a credentialed operation via the Vault. Carries operation parameters, permission_token, and agent-specified timeout_seconds. Vault deduplicates on request_id.
`aegis.orchestrator.state.write`	At-least-once	EDD §8.6	Request the Orchestrator to persist a state record via Memory/Storage.
`aegis.orchestrator.state.read.request`	At-least-once	EDD §8.6	Request the Orchestrator to retrieve state records from Memory/Storage.
`aegis.orchestrator.clarification.request`	At-least-once	EDD §8.7	Request the Orchestrator to surface a question to the user via User I/O.
`aegis.orchestrator.audit.event`	At-least-once	EDD §8.8	Security and audit event (incl. vault_execute_* event types). Orchestrator must store append-only.
`aegis.orchestrator.error`	At-least-once	EDD §8.8	Component-level error requiring Orchestrator awareness.

1.3 Requirements FROM Orchestrator

Task Flows

ID	Requirement	Rationale
ORC-REQ-01	The Orchestrator MUST publish task_spec events conforming exactly to the schema in EDD §8.3. Malformed events are rejected and must not be retried without correction.	Schema enforcement is the only way to guarantee safe downstream provisioning.
ORC-REQ-02	The Orchestrator MUST include required_skill_domains[] (min 1 item) in every task_spec. The Agents Component does not infer skill requirements.	Skill domain specification is the Orchestrator's responsibility; capability matching is ours.

ORC-REQ-03	The Orchestrator MUST NOT publish the same task_id more than once unless the previous attempt returned task.failed or task.result with status=timed_out.	Duplicate task_ids without status check will cause duplicate agent spawn.
ORC-REQ-04	The Orchestrator MUST consume from aegis.orchestrator.agent.status and update its own task tracking accordingly. It must not assume an agent is ACTIVE without having received a task.accepted event.	Stale Orchestrator state leads to misdirected task assignments.

Credential Lifecycle Delegation

ID	Requirement	Rationale
ORC-REQ-05	The Orchestrator MUST consume from aegis.orchestrator.credential.request (operations: authorize, revoke only — fetch removed in v1.5) and publish a response to aegis.agents.credential.response within 2 seconds p99 under normal load.	Credential latency directly blocks agent provisioning. Slowness here fails our 30s new-agent SLA.
ORC-REQ-06	The Orchestrator MUST ensure that credential lifecycle responses are transmitted only to the requesting Agents Component consumer group. No other component may receive credential.response events.	Credential tokens must not be observable to any other component.
ORC-REQ-07	The Orchestrator MUST process credential.request (operation=revoke) events even after the associated task_id is complete. Revocation must be non-blocking — it does not delay task closure.	Orphaned tokens are a security risk. Revocation must succeed asynchronously.
ORC-REQ-08	The Orchestrator MUST return status=denied (not an error) when the Credential Vault refuses authorization. The error_reason field must be human-interpretable but must not expose vault internals.	Clear denial status is required for the Agents Component to produce a meaningful task.failed event.

Vault Execute Delegation (NEW in v1.5)

These requirements replace ORC-REQ-09 from CIC v1.4, which required the Orchestrator to relay raw credential values. Raw credential values are no longer transmitted anywhere. The Vault executes operations internally.

ID	Requirement	Rationale
ORC-REQ-09 ◆ NEW	The Orchestrator MUST consume from aegis.orchestrator.vault.execute.request and route the request to the Credential Vault. The Vault must execute the operation and the Orchestrator MUST relay the result to aegis.agents.vault.execute.result within timeout_seconds + a reasonable routing overhead. If the Vault does not respond within this window, the Orchestrator MUST publish a result with status=timed_out.	A Vault execute result that never arrives causes the agent's deadline to fire with no clean error path.
ORC-REQ-10 ◆ NEW	The Orchestrator MUST relay Vault progress events (aegis.agents.vault.execute.progress) to the Agents Component as they arrive. These are at-most-once — the Orchestrator MUST NOT buffer or replay them.	Progress events are informational. Buffering them changes their semantics and defeats stall detection.

ORC-REQ-11 ◆ NEW	The Orchestrator MUST enforce that vault.execute.result events carry operation_result only — never a raw credential value. If the Vault returns a response containing credential material, the Orchestrator MUST reject it and return status=execution_error.	The entire purpose of the Vault execute model is that no credential value ever transits any boundary. This is a hard security invariant.
ORC-REQ-12 ◆ NEW	The Orchestrator MUST pass the request_id from vault.execute.request through to the Vault and echo it back in the result. This enables idempotent resubmission after agent crash recovery.	Crash recovery resubmits in-flight Vault operations by request_id. If the Vault loses the ID, the deduplication guarantee breaks.
ORC-REQ-13 ◆ NEW	The Orchestrator MUST restrict aegis.agents.vault.execute.result to the Agents Component consumer group only. No other component may subscribe to this topic.	Operation results may contain sensitive output. Restricting the consumer group is the same control applied to credential.response.

State Persistence Delegation

ID	Requirement	Rationale
ORC-REQ-14	The Orchestrator MUST consume from aegis.orchestrator.state.write and publish a write.ack to aegis.agents.state.write.ack within 1 second p99 under normal load. The ack must only be published after durability is confirmed by the Memory/Storage Component.	The Agents Component blocks on write.ack for critical operations. A false ack that precedes actual persistence is a data loss scenario.
ORC-REQ-15	The Orchestrator MUST enforce append-only semantics for state.write events where data_type=audit_log. UPDATE and DELETE operations on audit records must be rejected with status=rejected and rejection_reason=AUDIT_IMMUTABILITY_VIOLATION.	Audit immutability is a security control, not a suggestion.
ORC-REQ-16	The Orchestrator MUST enforce agent-scoped isolation on state.read.request events. Read results must be scoped strictly to the agent_id in the request — no cross-agent data leakage.	Cross-agent reads are a security violation and could contaminate agent context with irrelevant data (causing context rot).
ORC-REQ-17	The Orchestrator MUST replicate state records with data_type=snapshot to at least one peer node before publishing the write.ack. Read-your-own-writes consistency is required for snapshot recovery.	A recovery flow that cannot read its own most recent snapshot is unreliable.
ORC-REQ-18	The Orchestrator MUST honour TTL semantics. Records submitted with ttl_seconds must be eligible for garbage collection after expiry. Records with ttl_seconds=null must not be evicted.	TTL-based eviction is our primary memory management mechanism.

User I/O Delegation

ID	Requirement	Rationale
ORC-REQ-19	The Orchestrator MUST relay clarification.request events to the User I/O Component and return the user's response to aegis.agents.clarification.response within the timeout_seconds specified in the request.	A clarification that never arrives causes the Agents Component to time out the task.

ORC-REQ-20	The Orchestrator MUST route user session context (keyed on user_context_id from the task_spec) to the Memory/Storage Component before the Agents Component may request a state.read for it.	Agents reading user context before it is persisted get empty results and produce generic, uncustomised outputs.

1.4 Commitments TO Orchestrator

ID	Commitment	SLA
ORC-CMT-01	We will publish task.accepted or task.failed for every well-formed task_spec. We will never silently drop a task.	Within 5 seconds of receipt
ORC-CMT-02	We will publish task.result to aegis.orchestrator.task.result on task completion. task_id and user_context_id will always be echoed.	On completion, at-least-once
ORC-CMT-03	We will respond to capability_query with a capability.response: match, partial_match, or no_match.	Within 500ms p99
ORC-CMT-04	We will publish agent.status events for all critical state transitions (RECOVERING, TERMINATED, FAILED). The Orchestrator will not be surprised by agent disappearance.	Within 1 second of state change
ORC-CMT-05	We will publish one audit.event for every security-relevant operation, including vault_execute_request, vault_execute_result, and vault_execute_timeout events. The Orchestrator must store these append-only.	On each security event
ORC-CMT-06	We will never publish raw credential values to any topic. vault.execute.request carries only permission_token references and operation parameters — never the underlying secret. We consume operation results from vault.execute.result and pass them to agents. No credential value is ever present in this flow.	100% invariant
ORC-CMT-07	We will echo user_context_id from the task_spec in all outbound events for that task, enabling the Orchestrator to correlate outputs to the correct User I/O session without knowing agent internals.	100% fidelity
ORC-CMT-08	On agent recovery, we will resubmit in-flight vault.execute.request events using the same request_id. We will not generate a new request_id for recovered operations. This is safe because the Vault deduplicates on request_id.	100% invariant

1.5 Security Classification

Message / Data Element	Classification	Controls Applied
`task_spec payload`	INTERNAL — SENSITIVE	mTLS transport. Encrypted at rest in JetStream. Payload not logged at transport layer.
`task_result payload`	INTERNAL — SENSITIVE	Same as task_spec. May contain PII or confidential output.
`credential.request (authorize/revoke)`	INTERNAL — SENSITIVE	mTLS. Not persisted in stream beyond 1 hour. Logged by message_type only — payload excluded from transport logs.

`aegis.agents.credential.response`	INTERNAL — RESTRICTED	mTLS. Consumer group restricted to Agents Component only. No credential_value field (fetch removed in v1.5).
`aegis.orchestrator.vault.execute.request`	INTERNAL — SENSITIVE	mTLS. Not persisted beyond 1 hour. operation_params excluded from logs. permission_token treated as sensitive reference.
`aegis.agents.vault.execute.result`	INTERNAL — RESTRICTED	mTLS. Consumer group restricted to Agents Component only. operation_result excluded from audit logs. Never contains raw credential values.
`aegis.agents.vault.execute.progress`	INTERNAL — STANDARD	mTLS. At-most-once transient. Safe to log message metadata. No sensitive operation content.
`state.write (audit_log type)`	AUDIT — PROTECTED	mTLS. Append-only enforced at Orchestrator. Integrity hash on write. Separate read role for audit team.
`agent.status events`	INTERNAL — STANDARD	mTLS. Contains only agent_id and state. Safe to log for monitoring.
`clarification.request / response`	SENSITIVE — PII potential	mTLS. Not persisted beyond task lifetime.

Interface 2: Communications Component

Transport Infrastructure — NATS JetStream

2.1 Relationship Summary

The Communications Component is pure infrastructure. It does not interpret message content — it routes, delivers, persists, and guarantees events between the Agents Component and the Orchestrator. All topics defined in EDD §8.1 must be provisioned and enforced by the Communications Component before the Agents Component can operate.

v1.5 change: Three new Vault execution topics require stream provisioning. AGENTS_OUTBOUND_OPS is extended to include aegis.orchestrator.vault.execute.> and a new AGENTS_VAULT_INBOUND stream is added for the inbound vault.execute.progress and vault.execute.result topics.

2.2 Required Stream Configuration

Stream Name	Subject Filter	Retention	Replicas	Max Age	Notes

AGENTS_INBOUND	aegis.agents.task.> aegis.agents.capability.> aegis.agents.lifecycle.> aegis.agents.credential.> aegis.agents.state.> aegis.agents.clarification.>	WorkQueue (tasks) / Limits (others)	3 (quorum)	24h	All durable topics the Agents Component consumes from the Orchestrator. Does NOT include vault execute topics — see AGENTS_VAULT_INBOUND.
AGENTS_VAULT_INBOUND ◆ NEW	aegis.agents.vault.execute.progress aegis.agents.vault.execute.result	Limits (transient for progress; 1h for result)	3 (quorum)	1h (result) Transient (progress)	NEW v1.5. Separated from AGENTS_INBOUND because progress events are at-most-once transient while result events require at-least-once durability.
AGENTS_OUTBOUND_TASKS	aegis.orchestrator.task.>	Limits (7 day replay)	3 (quorum)	7 days	Task results and failures for Orchestrator processing.
AGENTS_OUTBOUND_OPS	aegis.orchestrator.credential.> aegis.orchestrator.vault.execute.> aegis.orchestrator.state.> aegis.orchestrator.clarification.>	WorkQueue (consumed once)	3 (quorum)	1h	Operational requests including vault.execute.request. Updated in v1.5 to add vault execute subject.
AGENTS_AUDIT	aegis.orchestrator.audit.event	Interest (permanent)	3 (quorum)	None — permanent	Append-only. Communications must block any non-Agents publisher.
AGENTS_STATUS	aegis.orchestrator.agent.status aegis.orchestrator.error	Limits (24h / 72h)	2	Per topic	Status and error broadcasts for Orchestrator monitoring.

2.3 Message Envelope

Every event published by the Agents Component must include the standard envelope defined in EDD §8.2. The Communications Component uses message_id for deduplication and source_component for access control enforcement. Enveloped events without a valid source_component='agents' claim on an aegis.orchestrator.* topic are rejected.

For vault.execute.request events, the correlation_id field MUST be set to the request_id of the Vault execute operation. This enables the Communications Component to correlate the result when it arrives on the return topic.

2.4 Access Control Requirements

ID	Requirement
COM-REQ-01	The Communications Component MUST enforce mTLS on all NATS connections. Connections without a certificate signed by the Aegis internal CA are refused.
COM-REQ-02	The Communications Component MUST enforce publish/subscribe access by client identity. The Agents Component's credentials authorize publish on aegis.orchestrator.* and subscribe on aegis.agents.* only. Cross-topic access is not permitted.
COM-REQ-03	The aegis.agents.credential.response topic MUST be restricted to Agents Component subscribers only. The Communications Component must enforce this at the stream consumer level — no other component may subscribe.
COM-REQ-04 ◆ NEW	The aegis.agents.vault.execute.result topic MUST be restricted to Agents Component subscribers only. No other component may subscribe. This mirrors the same restriction on credential.response. Operation results may contain sensitive data.
COM-REQ-05	The aegis.orchestrator.audit.event stream MUST be write-protected for all consumers. Once written, audit events must not be modifiable or deletable by any component including the Orchestrator.
COM-REQ-06	All JetStream stream storage MUST be encrypted at rest. Key management must integrate with OpenBao.
COM-REQ-07	The Communications Component MUST provide delivery receipts for at-least-once topics so the Agents Component can confirm critical events (task.result, audit.event, state.write, vault.execute.request) were durably persisted in the stream.
COM-REQ-08 ◆ NEW	For aegis.agents.vault.execute.progress, the Communications Component MUST NOT apply at-least-once guarantees. These events are explicitly at-most-once transient. If a progress event is lost, it must not be redelivered.

Interface 3: Credential Vault

Downstream — Vault-Executed Operations Model (v1.5)

v1.5 architectural change (ADR-004): The Credential Vault now executes credentialed operations on behalf of agents and returns operation results. Raw credential values never leave the Vault boundary. The previous Phase 2 (fetch — returning a raw credential value) has been replaced by the Vault Execute model. Phase 1 (authorize) and Phase 3 (revoke) are unchanged. These requirements are addressed to the Orchestrator team and the Credential Vault team jointly.

3.1 Relationship Summary

The Agents Component has no direct integration with the Credential Vault. All credential operations flow through the Orchestrator. In v1.5, this means two distinct coordination patterns:

1. Credential lifecycle (authorize, revoke): we publish a credential.request; the Orchestrator coordinates with the Vault; the Orchestrator returns a status result. This pattern is unchanged from v1.4.

2. Vault-executed operations: we publish a vault.execute.request with operation parameters and a permission_token; the Orchestrator routes it to the Vault; the Vault retrieves the credential internally, executes the operation, and publishes the result. The raw credential never leaves the Vault.

3.2 Three-Phase Credential Model

Phase	When	We Publish	Orchestrator + Vault Must Do	We Receive Back
Phase 1: Authorize	Agent spawn time	`credential.request` `{operation: authorize, skill_domains[], ttl_seconds}`	Ask Vault to create a scoped policy token for these domains. Return the token reference only — never the underlying secrets.	`credential.response` `{status: granted, permission_token, expires_at}`
Phase 2: Execute ◆ REPLACES FETCH	Runtime — skill invocation requiring credentialed operation	`vault.execute.request` `{operation_type, operation_params, permission_token, timeout_seconds, credential_type}`	Vault validates token scope internally. Retrieves the credential internally. Executes the operation. Publishes progress events during execution. Returns the operation result — never the raw credential value.	`vault.execute.result` `{status, operation_result, elapsed_ms}` `vault.execute.progress` `{progress_type, message, elapsed_ms}`
Phase 3: Revoke	Agent termination	`credential.request` `{operation: revoke, permission_token}`	Revoke the token in the Vault recursively (token + all child leases).	`credential.response` `{status: revoked}`

3.3 Requirements from Orchestrator and Vault (Credential Fulfilment)

ID	Requirement
VLT-REQ-01	The Orchestrator MUST create scoped Vault tokens using service token type (not batch tokens). Service tokens support per-agent revocation. This is an internal Orchestrator + Vault implementation requirement.
VLT-REQ-02	The Orchestrator MUST derive the Vault policy set from the skill_domains[] in our credential.request (operation=authorize). Policy names follow: aegis-agents-{domain}. If a domain has no corresponding policy, the Orchestrator must return status=denied — not silently issue an empty-permission token.

VLT-REQ-03	The Orchestrator MUST attach agent_id, task_id, and skill_domains[] as metadata to every Vault token issued. This enables audit correlation between our audit.event stream and the Vault's internal access logs.
VLT-REQ-04	The Orchestrator MUST enforce TTL on Vault tokens. The token lifetime must not exceed our requested ttl_seconds. Expired tokens must return status=denied immediately — no grace period.
VLT-REQ-05 ◆ NEW	The Vault MUST execute the credentialed operation internally using the credential associated with the validated permission_token. The credential value MUST NOT be returned to the Orchestrator or the Agents Component at any point during or after execution.
VLT-REQ-06 ◆ NEW	The Vault MUST validate the permission_token scope against the requested credential_type and operation_type before executing. If the token does not authorize the requested operation, the Vault MUST return status=scope_violation without executing and without accessing the secret.
VLT-REQ-07 ◆ NEW	The Vault MUST deduplicate vault.execute.request events on request_id. If a request_id has already been executed (or is in-progress), the Vault MUST return the cached result (or wait for the in-progress result) rather than executing again. This guarantees exactly-once execution semantics for recovered agents.
VLT-REQ-08 ◆ NEW	The Vault MUST enforce the timeout_seconds specified in vault.execute.request as a hard execution deadline. If the operation does not complete within this window, the Vault MUST return status=timed_out.
VLT-REQ-09 ◆ NEW	The Vault MUST publish progress events (at-most-once) during long-running operations. The progress event contract (time-based heartbeat interval, semantic milestones, or both) is documented in EDD OQ-10 and subject to joint agreement between the Agents Component team and the Vault team.
VLT-REQ-10 ◆ NEW	The operation_result field in vault.execute.result MUST contain only the output of the executed operation. It MUST NOT contain the credential value, vault path details, or any internal Vault state. Audit events for vault_execute_* events MUST NOT include operation_result — only metadata (request_id, operation_type, status, elapsed_ms).
VLT-REQ-11	The Orchestrator MUST process credential.request (operation=revoke) asynchronously and non-blockingly. It must not require the Agents Component to wait for revocation confirmation before considering the agent terminated.

3.4 Required Vault Policy Definitions

The Credential Vault team must implement the following internal policies before integration testing. In v1.5, these policies govern which operation_types may be executed under each policy — not just which secrets may be read.

Policy Name	Secret Paths Covered	Operations Permitted	Notes
`aegis-agents-web`	`aegis/{env}/web/*`	execute only	Web/HTTP credentialed operations. API keys used internally by Vault to execute requests.
`aegis-agents-data`	`aegis/{env}/data/*`	execute only	Database credential operations. Vault executes queries and returns results.
`aegis-agents-comms`	`aegis/{env}/comms/*`	execute only	Messaging service credential operations.

aegis-agents-storage	`aegis/{env}/storage/*`	execute only	Object and file storage credential operations.
aegis-agents-compute	`aegis/{env}/compute/*`	execute only	Cloud compute API credential operations.
aegis-agents-admin	`aegis/{env}/admin/*`	execute only	RESERVED. Only issued when Orchestrator explicitly sets admin_access=true in task. Not available by default.

All policies are execute-only from the Agents Component perspective. The Agents Component never receives the raw secret values — the Vault uses them internally to execute the requested operation. Any vault.execute.request requiring write access to Aegis-managed secrets indicates a design problem, not a permissions gap.

3.5 Security Levels for Credential Data

Level	Description	Applied To	Controls
L0 — Public	Non-sensitive capability metadata	Skill domain names	mTLS only
L1 — Internal Standard	Operational identifiers	agent_id, task_id, permission_token reference	mTLS + token binding
L2 — Restricted	Sensitive operation results in transit	operation_result in vault.execute.result	mTLS + consumer group restriction + operation_result excluded from all logs
L3 — Audit Protected	Credential and execute operation records	audit.event with event_type=credential_* or vault_execute_*	Append-only, integrity hash, separate audit read role. vault_execute audit records exclude operation_result.
L4 — Vault Internal	Raw credential values	All secret values in OpenBao	Never visible to Agents Component, Orchestrator, or any other component. Vault holds and uses credentials internally.

Interface 4: Memory / Storage Component

Downstream — Data Schemas, Memory Architecture, and Storage Requirements

4.1 Relationship Summary

The Agents Component has no direct integration with the Memory/Storage Component. All state reads and writes are routed through the Orchestrator via state.write and state.read.request events. This section specifies the data schemas, memory paradigms, and storage requirements we impose on the Orchestrator and Memory/Storage team.

Design principle: we treat memory as a retrieval problem, not a storage problem. We store the minimum necessary data in the most queryable structure. This directly combats context rot.

4.2 Memory Architecture: Three Paradigms

Memory Type	Analogy	What It Stores	Access Pattern	Retention
Episodic	Personal diary	Discrete events: task executions, state transitions, interactions, outcomes, vault execute completions. What happened, when, to whom.	Time-range queries. Agent-scoped. Specific episodes retrieved, not full history.	TTL-based. Default 90 days. Audit records: 1 year.
Semantic	Reference manual	Facts: skill definitions, agent capability profiles, domain knowledge. What is true.	Key-based lookup. Shared across agents of same type. Frequently read, rarely written.	Long-lived. No TTL unless explicitly invalidated. Skill cache: invalidated on skill update.
Procedural	Muscle memory	How-to patterns: successful task execution sequences, vault execute operation patterns, workflow templates.	Pattern-match retrieval. Shared across agents. Written on task success; read at provisioning.	Versioned. Old patterns superseded not deleted. Effective period tracked.

4.3 Table Definitions

Episodic Memory

agent_episodes — Task events and lifecycle records

Column	Type	Nullable	Index	Description
`episode_id`	TEXT (UUID)	NO	PRIMARY KEY	Unique identifier for this episode.
`agent_id`	TEXT (UUID)	NO	INDEX	Agent that generated this episode.
`episode_type`	TEXT (enum)	NO	INDEX	task_execution | state_transition | credential_event | skill_invocation | vault_execute | error_event | recovery_event | user_context
`task_id`	TEXT (UUID)	YES	INDEX	Associated task. Null for non-task events.
`timestamp_start`	TEXT (ISO 8601)	NO	INDEX	When the episode began.
`timestamp_end`	TEXT (ISO 8601)	YES		When the episode ended. Null if ongoing.
`outcome`	TEXT (enum)	YES		success | failure | timeout | cancelled | recovered
`payload_summary`	TEXT (JSON)	NO		Structured summary. NEVER raw transcript or operation results. Max 4KB.

skill_domains_used	TEXT (JSON array)	YES		Skill domains invoked during this episode.
error_code	TEXT	YES		Error code if outcome=failure. Null otherwise.
created_at	TEXT (ISO 8601)	NO		Row creation time. Used for TTL eviction.
ttl_expires_at	TEXT (ISO 8601)	YES		Garbage-collection eligibility time. Null = indefinite.

agent_state_snapshots — Recovery checkpoints

Column	Type	Nullable	Index	Description
snapshot_id	TEXT (UUID)	NO	PRIMARY KEY	Unique identifier.
agent_id	TEXT (UUID)	NO	INDEX	Agent this snapshot belongs to.
snapshot_type	TEXT (enum)	NO	INDEX	checkpoint | pre_suspend | pre_recovery | task_complete
task_id	TEXT (UUID)	YES	INDEX	In-progress task at snapshot time.
task_progress	TEXT (JSON)	YES		Structured task progress state.
skill_context	TEXT (JSON)	YES		Active skill domains and commands at snapshot time.
inflight_vault_requests	TEXT (JSON)	YES		NEW v1.5: List of {request_id, operation_type, status} for Vault execute operations in-flight at snapshot time. Used on recovery to determine which operations to resubmit.
timestamp	TEXT (ISO 8601)	NO	INDEX	When snapshot was taken.
is_latest	INTEGER (bool)	NO	INDEX	1 if most recent snapshot for this agent_id.

Semantic Memory

skill_definitions — The Skill Hierarchy tree

Column	Type	Nullable	Index	Description
skill_id	TEXT (UUID)	NO	PRIMARY KEY	Unique identifier for this skill node.
parent_skill_id	TEXT (UUID)	YES	INDEX	Parent node. Null for top-level domains.

`level`	INTEGER	NO	INDEX	1=domain, 2=command, 3=param_spec
`domain_id`	TEXT	NO	INDEX	Top-level domain (e.g. 'web', 'data').
`name`	TEXT	NO		Human-readable skill name.
`description`	TEXT	NO		Help text returned in --help queries. Max 200 chars.
`required_credential_types`	TEXT (JSON array)	YES		Credential types needed for this command. Level=2 nodes only. In v1.5, this determines which operations are routed through the Vault executor.
`param_schema`	TEXT (JSON)	YES		Full parameter JSON Schema. Level=3 nodes only.
`version`	INTEGER	NO		Incremented on update. Used for cache invalidation.
`is_active`	INTEGER (bool)	NO	INDEX	0=deprecated (retained for rollback). 1=active.
`updated_at`	TEXT (ISO 8601)	NO		Last modification timestamp.

agent_capability_profiles — Per-agent capability records

Column	Type	Nullable	Index	Description
`profile_id`	TEXT (UUID)	NO	PRIMARY KEY	Unique profile identifier.
`agent_id`	TEXT (UUID)	NO	INDEX, UNIQUE	One profile per agent.
`skill_domains`	TEXT (JSON array)	NO		Domains provisioned for this agent.
`known_skill_paths`	TEXT (JSON)	YES		domain/command paths previously invoked by this agent.
`performance_profile`	TEXT (JSON)	YES		Avg task duration, success rate, common failure modes.
`last_updated`	TEXT (ISO 8601)	NO		When last written.

Procedural Memory

execution_patterns — Successful task execution templates

Column	Type	Nullable	Index	Description
`pattern_id`	TEXT (UUID)	NO	PRIMARY KEY	Unique pattern identifier.

skill_domains	TEXT (JSON array)	NO	INDEX	Domains involved.
task_type_hint	TEXT	YES	INDEX	Optional task type label to narrow pattern retrieval.
skill_invocation_sequence	TEXT (JSON)	NO		Ordered list of { domain, command } pairs. Parameter values are NEVER stored — only structure.
vault_execute_sequence	TEXT (JSON)	YES		NEW v1.5: Ordered list of {operation_type, credential_type} pairs observed during successful execution. Used to optimise pre-authorization scope. No operation_result or credential values stored.
avg_execution_seconds	REAL	YES		Observed average execution time.
success_count	INTEGER	NO		Times this pattern produced a successful outcome.
is_current	INTEGER (bool)	NO	INDEX	One current version per skill_domains + task_type_hint combination.
last_observed_at	TEXT (ISO 8601)	NO		Most recent successful observation.

agent_audit_log — Append-only security event store

Written exclusively via state.write events with data_type=audit_log. Updates and deletes are rejected at the Orchestrator level. v1.5 adds vault_execute_* event types.

Column	Type	Nullable	Index	Description
log_id	TEXT (UUID)	NO	PRIMARY KEY	Unique log entry identifier.
agent_id	TEXT (UUID)	NO	INDEX	Agent that generated this event.
event_type	TEXT (enum)	NO	INDEX	state_transition | credential_grant | credential_deny | credential_revoke | scope_violation | vault_execute_request | vault_execute_result | vault_execute_timeout | provisioning_start | provisioning_complete | provisioning_fail | recovery_attempt | task_accepted | task_completed | task_failed
task_id	TEXT (UUID)	YES	INDEX	Associated task, if applicable.
request_id	TEXT (UUID)	YES	INDEX	NEW v1.5: For vault_execute_* events: the request_id of the Vault execute operation.
initiating_module	TEXT	NO		Which internal module generated this event.
outcome	TEXT (enum)	NO		success | denied | failed | partial

`event_detail`	TEXT (JSON)	NO		Structured event detail. No PII, no credential values, no operation_result. For vault_execute_*: request_id, operation_type, status, elapsed_ms only.
`timestamp`	TEXT (ISO 8601)	NO	INDEX	Immutable event time.

4.4 Requirements from Orchestrator (Storage Fulfilment)

ID	Requirement
MEM-REQ-01	The Orchestrator MUST process state.write events and return a write.ack only after durability is confirmed by the Memory/Storage Component. A false ack is a data loss scenario.
MEM-REQ-02	The Orchestrator MUST enforce schema validation on state.write payloads. Writes targeting unrecognised data_type values must return status=rejected with reason INVALID_DATA_TYPE.
MEM-REQ-03	The Orchestrator MUST enforce append-only semantics for data_type=audit_log. Any write event attempting to update or delete an audit record must return status=rejected with reason AUDIT_IMMUTABILITY_VIOLATION.
MEM-REQ-04	The Orchestrator MUST enforce agent-scoped isolation on state.read.request events. Read results must not include records from other agents, except for shared tables (skill_definitions, execution_patterns).
MEM-REQ-05	The Orchestrator MUST replicate state records with data_type=snapshot to at least one peer before publishing write.ack. Recovery flows depend on read-your-own-writes consistency. Snapshots must include the inflight_vault_requests field (NEW v1.5) for recovery to identify operations to resubmit.
MEM-REQ-06	The Orchestrator MUST honour TTL semantics. Records with ttl_seconds set must be eligible for garbage collection after expiry. Records with ttl_seconds=null must never be evicted. Audit records are exempt from eviction regardless of ttl_seconds.
MEM-REQ-07	All data stored on behalf of the Agents Component MUST be encrypted at rest using AES-256. Key management must integrate with OpenBao.
MEM-REQ-08	Schema changes to any table in §4.3 require joint written agreement between the Memory Component team, Orchestrator team, and Agents Component team, plus a versioned migration plan with zero-downtime rollout.

4.5 Security Classification

Table	Classification	Encryption	Access via Orchestrator
agent_episodes (payload_summary)	SENSITIVE	AES-256 at rest	Read/write: Agents Component requests only. No cross-agent reads.
agent_state_snapshots (incl. inflight_vault_requests)	SENSITIVE	AES-256 at rest	Read: recovery flows only, scoped to single agent_id.
skill_definitions	INTERNAL	Standard encryption	Read: any agent request. Write: Skill Hierarchy Manager write requests only, with version increment required.

agent_capability_profiles	INTERNAL	Standard encryption	Read/write: Agent Registry and Agent Factory requests only.
execution_patterns (incl. vault_execute_sequence)	INTERNAL	Standard encryption	Read: provisioning reads only. Write: post-task-success writes only. vault_execute_sequence never includes credential values or operation results.
agent_audit_log	AUDIT — PROTECTED	AES-256 + write-once integrity hash	Write: append only via Orchestrator. Read: Security/Audit team via dedicated role. vault_execute_* records exclude operation_result.

Interface 5: User I/O Component

Downstream — User Context, Clarification, Output Routing, and Feedback

5.1 Relationship Summary

The Agents Component has no direct integration with User I/O. User intent arrives in our task_spec (via the Orchestrator). User output is published in our task.result (the Orchestrator routes it to User I/O). User clarifications are requested via clarification.request events to the Orchestrator, which relays them to User I/O and returns responses. User context and feedback are stored/retrieved via state.write and state.read.request events to the Orchestrator.

The Agents Component never calls User I/O directly. We publish events to the Orchestrator with a user_context_id. The Orchestrator is responsible for associating these with the correct user session, routing questions to User I/O, and relaying responses back. We define what data we send; the Orchestrator and User I/O determine how it is displayed.

This interface is unchanged from CIC v1.4.

5.2 User Context in Agent Execution

Data Flow Step	Who Does It	Event / Action
User context written to storage	User I/O Component, before task dispatch	User I/O sends data to Orchestrator for persistence in agent_episodes (episode_type=user_context). The record must exist before the Orchestrator dispatches the task_spec to us.
We request the context	Agents Component, at task start if user_context_id is present	Publish aegis.orchestrator.state.read.request with { agent_id, data_type: episode, filters: { episode_type: user_context, key: user_context_id } }
Orchestrator fetches from Memory	Orchestrator, on receiving our read request	Retrieves the episode record from Memory/Storage and publishes the result to aegis.agents.state.read.response.

Agent primed with context	Agents Component, on receiving state.read.response	Context record is included in the agent's initial provisioning payload. Max 500 tokens. Structured JSON only — never raw transcript.

5.3 Output Routing via Orchestrator

Output Type	We Publish	Orchestrator Responsibility
Final task result	aegis.orchestrator.task.result with user_context_id echoed	Route result payload and user_message to the correct User I/O session.
Progress update	aegis.orchestrator.agent.status with task_id, user_context_id, and progress_summary	Stream progress to the user via User I/O as intermediate feedback.
Clarification needed	aegis.orchestrator.clarification.request with question and options[]	Surface the question to the user, collect the response, and return it on aegis.agents.clarification.response.
Task failure	aegis.orchestrator.task.failed with user_message field	Display user_message to the user. Never expose error_code or stack traces.

5.4 Feedback and Learning Loop

Feedback Type	How It Reaches Us	How We Use It
Explicit rating (thumbs up/down)	Orchestrator publishes state.write {data_type: execution_pattern_feedback, rating: 1|-1, task_id}	Update execution_patterns.success_count or flag pattern for review.
Implicit acceptance (no edit)	Orchestrator publishes state.write {data_type: execution_pattern_feedback, accepted: true, edit_distance: 0}	Reinforce the pattern used for this task.
High edit distance (user rewrote output)	Orchestrator publishes state.write {data_type: execution_pattern_feedback, edit_distance: N}	Flag the pattern as needing improvement. N > threshold triggers pattern review.
Preference update	Orchestrator updates user_context record in Memory/Storage	We read updated context on next task for this user via state.read.request.

5.5 Requirements from Orchestrator (User I/O Fulfilment)

ID	Requirement

UIO-REQ-01	The Orchestrator MUST ensure that user session context (episode_type=user_context, keyed on user_context_id) is persisted in Memory/Storage BEFORE the Orchestrator dispatches the task_spec to the Agents Component. We cannot read context that does not yet exist.
UIO-REQ-02	User session context payload_summary MUST NOT include raw conversation transcripts. Only structured JSON summaries with a maximum of 500 tokens are acceptable. Raw transcripts cause context rot.
UIO-REQ-03	The Orchestrator MUST relay clarification responses within the timeout_seconds specified in the clarification.request. If the user does not respond in time, the Orchestrator must return status=timed_out — it must not simply not respond.
UIO-REQ-04	The Orchestrator MUST route our task.result and task.failed events to the correct User I/O session using the user_context_id we echo in every outbound event. No additional routing information will be provided by the Agents Component.
UIO-REQ-05	The Orchestrator MUST publish feedback write events to our state topic within 60 seconds of the user interaction occurring.

5.6 Security Classification

Data Element	Classification	Controls
user_context record (preferences, goals)	SENSITIVE — PII	AES-256 at rest. Agent-scoped reads. Never logged at transport layer. TTL: session + 30 days.
task_result with user content	SENSITIVE	At-least-once delivery. Not persisted in stream beyond 7 days. Agent-scoped.
clarification.request / response	SENSITIVE — PII potential	mTLS. Not persisted beyond task lifetime.
feedback events	INTERNAL	Comments may contain PII. TTL: 90 days. Not linked to user PII in storage — keyed on task_id only.

Interface Contract Summary

The following table consolidates ratification requirements. All ratifications are PENDING — implementation begins after sign-off.

Component	Key Requirements FROM Them	Key Commitments TO Them	Status
Orchestrator	All 21 topics provisioned (incl. 3 new vault execute topics); credential lifecycle delegation within SLA; Vault execute routing within timeout; audit events stored append-only; vault.execute.result restricted to Agents Component; scope violations never result in secret or result exposure; user context available before task dispatch	One outbound event per inbound task; user_context_id echoed on all events; one clarification per task maximum; no raw credentials ever published; recovery resubmissions use original request_id; all audit events include initiating_module and request_id for vault_execute_* events	PENDING

Communications	mTLS on all connections; topic-level access controls; credential.response and vault.execute.result both restricted to Agents subscribers only; all 6 streams provisioned (incl. new AGENTS_VAULT_INBOUND); audit stream write-protected; vault.execute.progress delivered at-most-once only	Standard message envelope on all publishes (source_component='agents'); ACK on all at-least-once topics; startup topic-existence check before serving traffic	PENDING
Credential Vault	Service token type implemented; scoped policies in §3.4 all implemented as execute-only; token metadata support; recursive revocation; strict TTL enforcement; Vault executes operations internally and returns results only; Vault deduplicates on request_id; progress events published per OQ-10 agreement; raw credential values never transmitted	No raw credential caching by Agents Component; revoke request on every agent termination; scope violations blocked internally before escalation; vault.execute.request resubmitted with same request_id after crash	PENDING
Memory/ Storage	All 7 tables in §4.3 implemented (incl. inflight_vault_requests in snapshots, vault_execute_sequence in execution_patterns, request_id in audit_log); atomic writes; audit_log append-only at storage level; snapshot replication before ack; TTL eviction; AES-256 at rest	Structured tagged payloads only; no raw transcripts; no operation_result in any stored record; explicit TTLs on all writes; data_type always specified; request_id correlation on all reads	PENDING
User I/O	User context persisted before task dispatch; structured JSON context (no transcripts, max 500 tokens); feedback routed within 60s; clarification responses delivered within timeout or timed_out returned	user_context_id echoed on all events; human-readable user_message on failures; structured options[] in clarification requests; one clarification per task maximum	PENDING

Document Revision History

Version	Date	Author	Description
1.0	February 21, 2026	Design Team	Initial CIC. Interface contracts for all 5 components. Table definitions, memory paradigms, security levels.
1.2–1.3	February 21–25, 2026	Design Team	Orchestrator-as-hub: Interface 1 expanded to comprehensive bilateral contract. Interfaces 2–5 as downstream requirements.
1.4	February 25, 2026	Design Team	No structural changes. Split into standalone document for distribution to partner teams.

1.5	March 2026	Design Team	Vault execute model (ADR-004): removed fetch operation from credential.request; added vault.execute.request/progress/result topic trio; updated Orchestrator requirements (ORC-REQ-09–13 new, numbering shifted); added ORC-CMT-08; updated Communications streams (new AGENTS_VAULT_INBOUND stream, COM-REQ-04 and COM-REQ-08 new); rewrote Credential Vault interface §3 (phases, requirements, policy table); updated audit_log event types; added inflight_vault_requests to agent_state_snapshots; added vault_execute_sequence to execution_patterns; added request_id to agent_audit_log; updated security classification tables throughout.